



Universidad Nacional Experimental De Guayana.

Vicerrectorado Académico.

Coordinación General De Pregrado.

Proyecto De Carrera: Ingeniería en informática.

Unidad Curricular: Lenguajes y Compiladores.

Sección 2.

Lenguajes y Gramáticas

Formales

Profesor:

Márquez Félix

Alumnos:

Castro Robert V-30.994.039

Flores Endrys V-30.451.556

Ramírez Alexmary V-31.809.930

Vallenilla Daniel V-31.159.105

Ciudad Guayana, Junio de 2026.

Introducción

El procesamiento de lenguajes formales constituye uno de los pilares fundamentales en la formación de un ingeniero en informática, especialmente cuando se aborda el diseño e implementación de compiladores e intérpretes. El presente informe se desarrolla en el marco del Tema 3 de la unidad curricular Lenguaje y Compiladores, titulado "Lenguajes y Gramáticas Formales", y tiene como propósito central explorar las herramientas teóricas necesarias para modelar, validar y procesar lenguajes de manera rigurosa.

Para lograr dicho propósito, el informe se organiza en cuatro ejes principales. En primer lugar, se abordan los fundamentos de la Jerarquía de Chomsky y la relación entre gramáticas y lenguajes formales, estableciendo cómo un conjunto finito de reglas de producción es capaz de generar un lenguaje completo. En segundo lugar, se presenta un caso práctico de modelado mediante gramáticas libres de contexto aplicado a un alfabeto simbólico específico, demostrando el proceso de derivación para la generación de figuras geométricas. En tercer lugar, se analizan las patologías más comunes en el diseño de gramáticas —ambigüedad, recursividad por la izquierda y falta de factorización— y se proponen técnicas para su corrección, aspecto fundamental para garantizar la robustez de un compilador. Finalmente, se aborda el diseño de expresiones regulares y autómatas finitos determinísticos para el reconocimiento de un subconjunto del lenguaje PGN utilizado en notación de ajedrez.

Cada sección ha sido desarrollada atendiendo a los criterios de rigor teórico y aplicabilidad práctica exigidos en la guía de investigación, con el fin de demostrar no solo la comprensión de los conceptos, sino también la capacidad de implementarlos en escenarios concretos.

2.1. Relación Gramática-Lenguaje

2.1.1. Relación Gramática-Lenguaje

Concepto de Alfabeto

Un **alfabeto** es un conjunto finito y no vacío de símbolos elementales que sirven como unidad básica para la construcción de cadenas o palabras dentro de un lenguaje formal. Se denota generalmente con la letra griega Σ (sigma mayúscula). La condición de ser "finito" es fundamental, pues de lo contrario se tendría una cantidad infinita de símbolos básicos, lo que haría imposible definir algoritmos de reconocimiento o generación. La condición de ser "no vacío" garantiza que exista al menos un símbolo con el cual construir palabras, aunque existe el concepto de lenguaje vacío que no contiene ninguna palabra, pero el alfabeto subyacente siempre debe tener al menos un símbolo. Por ejemplo:

- $\Sigma = \{a, b\}$
- $\Sigma = \{0, 1\}$
- $\Sigma = \{a, b, c, \dots, z\}$

No existe un orden implícito en los símbolos de un alfabeto; son simplemente elementos distinguibles.

Concepto de Palabra (o Cadena)

Una **palabra** (también llamada cadena o string) es una secuencia finita de símbolos pertenecientes a un alfabeto Σ . El término "secuencia" implica que el orden de los símbolos importa: la palabra "ab" es diferente de la palabra "ba", aunque utilicen los mismos símbolos. El término "finita" es esencial, pues si se permitieran secuencias infinitas, se entraría en el terreno de los lenguajes infinitarios, que no son de interés práctico en la teoría de lenguajes formales estándar.

La **longitud** de una palabra es el número de símbolos que la componen. Por ejemplo, sobre el alfabeto $\Sigma = \{a, b\}$, la palabra "ab" tiene longitud 2, la palabra "aaab" tiene longitud 4, y la palabra "a" tiene longitud 1. Existe un caso especial: la **palabra vacía**, denotada como ϵ (épsilon), que es la palabra de longitud cero, es decir, no contiene ningún símbolo. La palabra vacía pertenece

a todo alfabeto y es un elemento fundamental porque permite la terminación de procesos recursivos en las gramáticas.

Ejemplos sobre $\Sigma=\{a,b\}$:

- a (longitud 1)
- ab (longitud 2)
- aab (longitud 3)
- ε (longitud 0)

El conjunto de **todas** las palabras posibles sobre un alfabeto Σ , incluyendo la palabra vacía, se denota como Σ^* . El operador $*$ se conoce como **cerradura de Kleene**. Si se excluye la palabra vacía, se denota como Σ^+ .

Concepto de Lenguaje Formal

Un **lenguaje formal** L sobre un alfabeto Σ es cualquier subconjunto de Σ^* . Es decir:

$$L \subseteq \Sigma^*$$

Un lenguaje formal puede ser:

- Finito: contiene un número limitado de palabras (ej: $L=\{a,aa,aaa\}$)
- Infinito: contiene un número ilimitado de palabras (ej: $L=\{anbn|n \geq 0\}$, que genera $\varepsilon, ab, aabb, aaabbb, \dots$)
- Vacío: no contiene ninguna palabra ($L=\emptyset$)
- Universal: contiene todas las palabras sobre Σ ($L=\Sigma^*$)

Concepto de Gramática Formal

Una **gramática formal** es un dispositivo matemático que permite **generar** un lenguaje formal. Se define como una cuádrupla:

$$G=(VN,VT,S,P)$$

Componente	Notación	Descripción	Ejemplo
No terminales	VN	Conjunto finito de símbolos que representan categorías o estructuras. No aparecen en las palabras finales.	$\{S,A,B\}$
Terminales	VT	Conjunto finito de símbolos que constituyen las palabras del lenguaje. Coincide con el alfabeto Σ	$\{a,b\}$
Símbolo inicial	S	Un no terminal especial que da inicio a todas las derivaciones. $S \in VN$	S
Producciones	P	Conjunto finito de reglas de reescritura de la forma $\alpha \rightarrow \beta$, donde $\alpha \in (VN \cup VT)^+$ (contiene al menos un no terminal) y $\beta \in (VN \cup VT)^*$	$S \rightarrow aSb, S \rightarrow \epsilon$

Relación formal entre Gramática y Lenguaje

Dada una gramática G , el **lenguaje generado por G** se define como:

$$L(G) = \{w \in V_T^* \mid S \Rightarrow^* w\}$$

Esto se lee: "El conjunto de todas las palabras w compuestas exclusivamente por símbolos terminales tales que existe una derivación desde el símbolo inicial S hasta w en cero o más pasos".

El símbolo $\Rightarrow^*$ indica la **cerradura reflexiva y transitiva** de la relación de derivación $\Rightarrow$, es decir, cero o más aplicaciones de reglas.

El mecanismo de derivación en detalle

La derivación es el proceso dinámico mediante el cual una gramática va construyendo palabras. Existen dos tipos principales de derivación:

1. **Derivación más a la izquierda (leftmost derivation):** En cada paso, se reescribe el no terminal más a la izquierda.
2. **Derivación más a la derecha (rightmost derivation):** En cada paso, se reescribe el no terminal más a la derecha.

Ejemplo completo de derivación:

Gramática para el lenguaje $anbn$:

- $VN = \{S\}$
- $VT = \{a, b\}$
- $P = \{S \rightarrow aSb \mid \varepsilon\}$

Derivación más a la izquierda para $a^3b^3 = aaabbb$: $a^3b^3 \Rightarrow aaabbb$:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaasbbb \Rightarrow aaabbb$

En cada paso:

- Paso 1: S se reemplaza por aSb
- Paso 2: el nuevo S se reemplaza nuevamente por aSb
- Paso 3: se repite
- Paso 4: el último S se reemplaza por ε

El resultado final es $aaabbb$, que pertenece a $L(G)$.

Comparativa: Generación vs. Reconocimiento

La relación entre gramáticas y lenguajes formales puede abordarse desde dos perspectivas complementarias que, aunque opuestas en su dirección, son matemáticamente equivalentes en muchos casos. La primera perspectiva es la generación, que parte de la gramática y aplica reglas de producción para construir cadenas válidas. La segunda perspectiva es el reconocimiento, que parte de una cadena cualquiera y busca determinar si existe una secuencia de derivaciones que la produzca desde el símbolo inicial.

Ambas perspectivas son esenciales en la construcción de compiladores. Por un lado, el programador genera código fuente siguiendo las reglas del lenguaje. Por otro lado, el compilador debe reconocer ese código y decidir si es estructuralmente correcto. Esta dualidad está formalizada por los teoremas de equivalencia: toda gramática regular tiene un autómata finito equivalente, toda gramática libre de contexto tiene un autómata de pila equivalente, y así sucesivamente.

Es importante distinguir dos perspectivas complementarias

Perspectiva	Definición	Mecanismo
Generación	Dada una gramática, produce todas las cadenas que pertenecen al lenguaje.	Derivaciones $S \Rightarrow^* w S \Rightarrow^* w$
Reconocimiento	Dada una cadena, determina si es válida dentro del lenguaje.	Autómatas, parsers, algoritmos de decisión

Una misma familia de lenguajes puede ser descrita tanto por gramáticas (generadores) como por autómatas (reconocedores). Esta dualidad es fundamental en la teoría de la computación y está formalizada por el **Teorema de equivalencia** entre gramáticas regulares y autómatas finitos, gramáticas libres de contexto y autómatas de pila, etc.

Propiedades de la relación gramática-lenguaje

La relación entre una gramática y el lenguaje que esta genera no es arbitraria, sino que obedece a ciertas propiedades fundamentales que cualquier estudiante de lenguajes formales debe conocer. Estas propiedades permiten comprender los límites y las capacidades de las gramáticas como herramientas generadoras, así como las relaciones que pueden establecerse entre diferentes gramáticas y sus respectivos lenguajes.

A continuación, se resumen estas propiedades en la siguiente tabla:

Propiedad	Explicación
Finitud	Una gramática con un número finito de producciones puede generar lenguajes infinitos gracias a la recursividad.
No unicidad	Distintas gramáticas pueden generar el mismo lenguaje; a estas se les denomina gramáticas equivalentes.
Inclusión	Si G_1 y G_2 son gramáticas, puede ocurrir que $L(G_1) \subseteq L(G_2)$
Indecidibilidad	Para gramáticas de Tipo 0, no existe un algoritmo general que determine si dos gramáticas distintas generan el mismo lenguaje.

Estas propiedades no solo describen el comportamiento teórico de las gramáticas, sino que tienen implicaciones directas en el diseño de compiladores y lenguajes de programación. La recursividad explica por qué un programa puede tener una cantidad ilimitada de instrucciones anidadas. La equivalencia entre gramáticas permite transformar una gramática ambigua en una

no ambigua sin alterar el lenguaje reconocido. La inclusión justifica por qué un analizador léxico (regular) no puede reconocer estructuras sintácticas (libres de contexto), y la indecidibilidad recuerda que existen límites fundamentales en lo que puede automatizarse: hay preguntas sobre lenguajes de programación que simplemente no pueden responderse con un algoritmo general.

2.1.2. Jerarquía de Chomsky

Contexto histórico

En 1956, el lingüista estadounidense **Noam Chomsky** publicó el artículo "Three models for the description of language", donde propuso una clasificación de las gramáticas formales que revolucionó tanto la lingüística teórica como la informática. Chomsky observó que los lenguajes naturales (como el español o el inglés) tienen estructuras jerárquicas que pueden ser modeladas con diferentes tipos de reglas gramaticales, y que estas reglas requieren diferentes capacidades computacionales para ser procesadas.

La jerarquía no solo clasifica gramáticas, sino que establece una relación directa entre:

- La **forma** de las reglas de producción
- El **poder expresivo** de los lenguajes generados
- La **complejidad computacional** del reconocimiento
- El **modelo de autómatas** necesario para reconocerlos

Estructura general de la jerarquía

La jerarquía es **inclusiva** (estrictamente anidada):

$$\text{Tipo 3} \subset \text{Tipo 2} \subset \text{Tipo 1} \subset \text{Tipo 0}$$

Es decir:

- Todo lenguaje regular es también libre de contexto (pero no al revés)
- Todo lenguaje libre de contexto es también sensible al contexto (pero no al revés)
- Todo lenguaje sensible al contexto es también recursivamente enumerable (pero no al revés)

Las inclusiones son **estrictas**, lo que significa que existen lenguajes en cada nivel que no pertenecen al nivel inferior.

Tipo 3: Gramáticas Regulares

Definición formal

Una gramática regular (por la derecha) es aquella cuyas producciones cumplen **exclusivamente** una de las dos formas siguientes:

1. $A \rightarrow aB$
2. $A \rightarrow a$
3. $A \rightarrow \varepsilon$ (opcional, para generar la cadena vacía)

Donde:

- $A, B \in VN$ (no terminales)
- $a \in VT$ (terminal)

También existen gramáticas regulares por la izquierda ($A \rightarrow Ba$), pero ambas definiciones son equivalentes en poder expresivo.

Poder expresivo

Las gramáticas regulares generan los **lenguajes regulares**, que son aquellos que pueden ser reconocidos por un **autómata finito** (determinístico o no determinístico). Estos lenguajes tienen la propiedad fundamental de poder ser descritos mediante **expresiones regulares**.

Limitaciones

Un lenguaje regular **no puede** contar ni recordar cantidades arbitrarias. Por ejemplo, el lenguaje $L = \{anbn | n \geq 0\}$ (mismo número de 'a's que de 'b's) **no es regular**. Un autómata finito tiene memoria limitada y no puede llevar la cuenta de cuántas 'a's ha leído para luego poner la misma cantidad de 'b's.

Tipo 2: Gramáticas Libres de Contexto (Context-Free Grammars - CFG)

Definición formal

Una gramática libre de contexto es aquella cuyas producciones tienen la forma:

$$A \rightarrow \gamma$$

Donde:

- $A \in VN$ (un único no terminal en el lado izquierdo)
- $\gamma \in (VN \cup V^T)^*$ (cualquier cadena de terminales y no terminales, puede ser vacía)

Poder expresivo

Las gramáticas libres de contexto generan los **lenguajes libres de contexto**, que pueden ser reconocidos por un **autómata de pila (PDA)**. La pila permite "recordar" cantidades arbitrarias mediante un mecanismo de apilamiento y desapilamiento.

Capacidades adicionales

A diferencia de los lenguajes regulares, los lenguajes libres de contexto sí pueden manejar:

- Estructuras anidadas (paréntesis balanceados)
- Conteo arbitrario ($anbn$)
- Construcciones recursivas

Limitaciones

Una gramática libre de contexto **no puede** expresar dependencias que requieren contexto. Por ejemplo, el lenguaje $L = \{anbnc^n \mid n \geq 0\}$ (igual número de 'a's, 'b's y 'c's) **no es libre de contexto**. Tampoco puede verificar que una variable haya sido declarada antes de usarse (eso requiere contexto).

Tipo 1: Gramáticas Sensibles al Contexto (Context-Sensitive Grammars - CSG)

Definición formal

Una gramática sensible al contexto es aquella cuyas producciones tienen la forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

Donde:

- $A \in VN$ (un no terminal)
- $\alpha, \beta \in (VN \cup VT)^*$ (contexto izquierdo y derecho)
- $\gamma \in (VN \cup VT)^+$ (cadena no vacía)

La regla **solo puede aplicarse** si el no terminal A aparece rodeado por el contexto α a la izquierda y β a la derecha.

Poder expresivo

Las gramáticas sensibles al contexto generan los **lenguajes sensibles al contexto**, que pueden ser reconocidos por un **autómata linealmente acotado (LBA)** (una máquina de Turing con memoria proporcional al tamaño de la entrada).

Capacidades adicionales

Pueden expresar lenguajes que requieren múltiples conteos simultáneos, como $anbncn$.

Limitaciones

El reconocimiento de lenguajes sensibles al contexto es computacionalmente costoso (problema PSPACE-completo), por lo que en la práctica los compiladores no implementan gramáticas Tipo 1 completas, sino que manejan el contexto mediante tablas de símbolos en la fase semántica.

Tipo 0: Gramáticas Sin Restricciones (Unrestricted Grammars)

Definición formal

Una gramática sin restricciones es aquella cuyas producciones tienen la forma:

$$\alpha \rightarrow \beta$$

Donde:

- $\alpha \in (V \cup V^T)^+$ (cadena no vacía que contiene al menos un no terminal)
- $\beta \in (V \cup V^T)^*$ (cualquier cadena, puede ser vacía)

No hay ninguna restricción adicional. El lado izquierdo puede tener múltiples símbolos y puede modificar el contexto de manera arbitraria.

Poder expresivo

Las gramáticas sin restricciones generan los **lenguajes recursivamente enumerables**, que son exactamente los lenguajes que puede reconocer una **Máquina de Turing**. Esto significa que cualquier problema computable (que pueda resolver un ordenador, dado tiempo y memoria infinitos) puede ser expresado como un lenguaje Tipo 0.

Limitaciones fundamentales

Para los lenguajes Tipo 0, el **problema de pertenencia** (determinar si una cadena dada pertenece al lenguaje) es **semi-decidible**: existe una Máquina de Turing que se detiene y acepta si la cadena pertenece al lenguaje, pero si no pertenece, puede que nunca se detenga (o puede que se detenga y rechace). Esto es consecuencia del **problema de la parada** (Halting Problem).

Aplicación en compiladores: En la práctica, los compiladores no usan gramáticas Tipo 0 porque el reconocimiento es indecidible en el caso general. Sin embargo, la generación de código y las optimizaciones avanzadas tienen poder de Turing completo.

2.2. Derivación y Modelado (Caso Práctico: Genoma)

2.2.1. Gramática para dibujar

El alfabeto utilizado es $\Sigma = \{a, c, g, t\}$. Estos cuatro símbolos representan las bases nitrogenadas del ADN: a para Adenina, c para Citosina, g para Guanina y t para Timina. La idea es usar estos símbolos como instrucciones para generar cadenas que representan figuras geométricas mediante una gramática libre de contexto.

Definición formal:

- V_N (No terminales): $\{D, L, R, C, T, H\}$
- V_T (Terminales): $\{a, c, g, t\}$
- S (Símbolo inicial): D

Producciones P :

Las producciones P son las siguientes:

- $D \rightarrow L \mid R \mid C \mid T \mid H$
- $L \rightarrow a L a \mid c L c \mid g L g \mid t L t \mid \epsilon$ (para generar líneas rectas)
- $R \rightarrow a R a \mid c R c \mid \epsilon$ (para generar ramas)
- $C \rightarrow a C c \mid \epsilon$ (para generar cuadrados)
- $T \rightarrow a T a \mid c T g \mid g T t \mid \epsilon$ (para generar árboles)
- $H \rightarrow a H a \mid c H c \mid g H g \mid t H t \mid \epsilon$ (para generar cubos)

A continuación, se presentan cinco ejemplos de derivación para diferentes figuras.

Ejemplo 1: Cuadrado

Para dibujar un cuadrado se utiliza el símbolo C . Se desea generar la cadena "aacc". La derivación es la siguiente:

$$C \Rightarrow aCc \Rightarrow aaCcc \Rightarrow aacc$$

Explicación: Se comienza con C . Se aplica la regla $C \rightarrow aCc$, obteniendo aCc . Se aplica nuevamente $C \rightarrow aCc$, obteniendo $aaCcc$. Finalmente se aplica $C \rightarrow \varepsilon$, eliminando C y dejando $aacc$. Esta cadena representa un cuadrado.

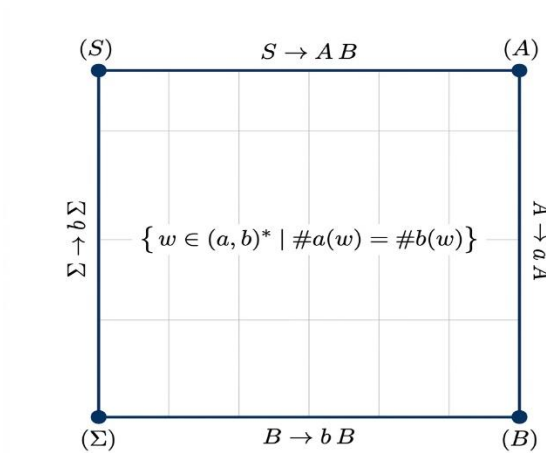


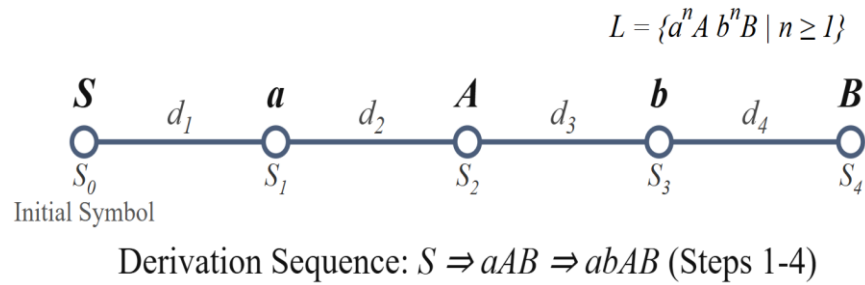
Fig. 1: Formal Language Derivation for CFL L_1 .

Ejemplo 2: Línea recta horizontal

Para dibujar una línea recta se utiliza el símbolo L . Se desea generar la cadena "aaaa". La derivación es la siguiente:

$$L \Rightarrow aLa \Rightarrow aaLaa \Rightarrow aaaa$$

Explicación: Se comienza con L . Se aplica $L \rightarrow aLa$, obteniendo aLa . Se aplica nuevamente $L \rightarrow aLa$, obteniendo $aaLaa$. Finalmente se aplica $L \rightarrow \varepsilon$, eliminando L y dejando $aaaa$. Esta cadena representa una línea recta de cuatro puntos.

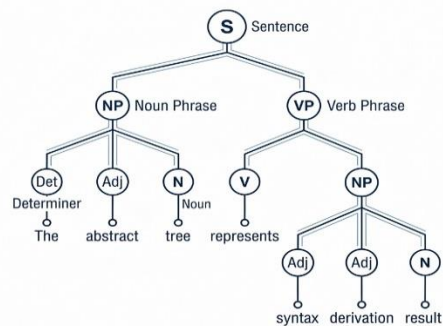


Ejemplo 3: Árbol con varias ramas y hojas

Para dibujar un árbol se utiliza el símbolo T. Se desea generar la cadena "aacggt". La derivación es la siguiente:

$$T \Rightarrow aTa \Rightarrow aaTa \Rightarrow aacTga \Rightarrow aacgTtga \Rightarrow aacggtaa$$

Explicación: Se comienza con T. Se aplica $T \rightarrow aTa$, obteniendo aTa. Se aplica nuevamente $T \rightarrow aTa$, obteniendo aaTaa. Se aplica $T \rightarrow cTg$, obteniendo aacTgaa. Se aplica $T \rightarrow gTt$, obteniendo aacgTtga. Finalmente se aplica $T \rightarrow \epsilon$, eliminando T y dejando aacggtaa. Nótese que el resultado final tiene una longitud de 8 caracteres: a a c g g t a a.

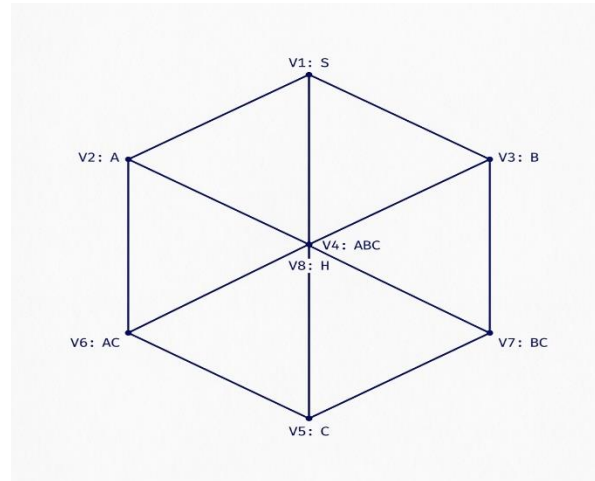


Ejemplo 4: Cubo

Para dibujar un cubo se utiliza el símbolo H. Se desea generar la cadena "aacctt". La derivación es la siguiente:

$$H \Rightarrow aHa \Rightarrow aaHaa \Rightarrow aacHcaa \Rightarrow aaccHccaa \Rightarrow aacctHtccaa \Rightarrow aaccttaa$$

Explicación: Se comienza con H. Se aplica $H \rightarrow aHa$, obteniendo aHa. Se aplica nuevamente $H \rightarrow aHa$, obteniendo aaHaa. Se aplica $H \rightarrow cHc$, obteniendo aacHcaa. Se aplica $H \rightarrow cHc$ nuevamente, obteniendo aaccHccaa. Se aplica $H \rightarrow tHt$, obteniendo aacctHtccaa. Finalmente se aplica $H \rightarrow \varepsilon$, eliminando H y dejando aaccttaa.

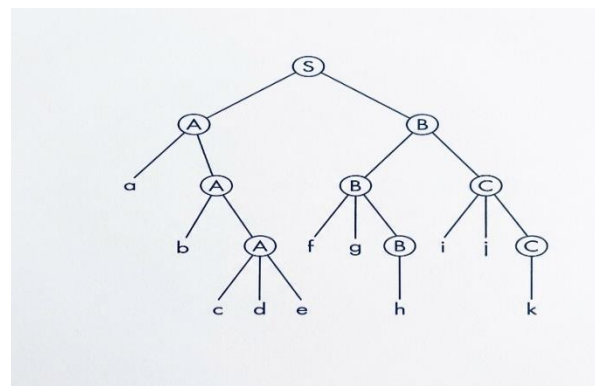


Ejemplo 5: Figura compuesta por ramas

Para dibujar una figura con ramas se utiliza el símbolo R. Se desea generar la cadena "acca". La derivación es la siguiente:

$$R \Rightarrow aRa \Rightarrow acRca \Rightarrow acca$$

Explicación: Se comienza con R. Se aplica $R \rightarrow aRa$, obteniendo aRa. Se aplica $R \rightarrow cRc$, obteniendo acRca. Finalmente se aplica $R \rightarrow \varepsilon$, eliminando R y dejando acca.



2.3. Higiene y Optimización de Gramáticas

2.3.1. Patologías de las Gramáticas

Las gramáticas mal diseñadas causan errores en los compiladores. A continuación se presentan tres casos prácticos diferentes.

Caso a) Gramática Ambigua

Una gramática es ambigua cuando existe al menos una cadena que puede ser generada por dos árboles de derivación distintos. Esto es problemático porque un compilador no sabría cuál estructura sintáctica elegir.

Ejemplo de gramática ambigua para expresiones aritméticas:

$$\langle E \rangle \rightarrow \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid \text{id}$$

Esta gramática es ambigua porque la cadena "id + id * id" tiene dos árboles de derivación diferentes.

Primer árbol de derivación (interpretación: (id + id) * id):

$$E \Rightarrow E + E \Rightarrow \text{id} + E \Rightarrow \text{id} + E * E \Rightarrow \text{id} + \text{id} * E \Rightarrow \text{id} + \text{id} * \text{id}$$

Segundo árbol de derivación (interpretación: id + (id * id)):

$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow \text{id} + E * E \Rightarrow \text{id} + \text{id} * E \Rightarrow \text{id} + \text{id} * \text{id}$$

La misma cadena "id + id * id" puede interpretarse de dos formas distintas, lo que demuestra la ambigüedad. Para corregirla, se debe definir una gramática no ambigua que respete la precedencia de operadores.

Gramática corregida no ambigua:

- $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle \mid \langle T \rangle$
- $\langle T \rangle \rightarrow \langle T \rangle * \langle F \rangle \mid \langle F \rangle$
- $\langle F \rangle \rightarrow \text{id}$

Caso b) Recursividad por la Izquierda

La recursividad por la izquierda ocurre cuando una producción tiene la forma $A \rightarrow A\alpha$. Esto causa problemas en los analizadores sintácticos descendentes (como los recursivos predictivos o LL(1)), porque generan un bucle infinito.

Ejemplo de gramática con recursividad por la izquierda:

- $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle \mid \langle T \rangle$
- $\langle T \rangle \rightarrow \langle T \rangle * \langle F \rangle \mid \langle F \rangle$
- $\langle F \rangle \rightarrow \text{id}$

Algoritmo paso a paso para eliminar la recursividad por la izquierda:

Para cada no terminal A que sea recursivo por la izquierda:

1. Separar las producciones de A en dos grupos: las recursivas por la izquierda ($A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots$) y las no recursivas ($A \rightarrow \beta_1 \mid \beta_2 \mid \dots$)
2. Crear un nuevo no terminal A'
3. Reemplazar todas las producciones de A por: $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots$
4. Definir las producciones de A' como: $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon$

Aplicando el algoritmo a la gramática anterior:

Para $\langle E \rangle$:

- producciones recursivas: $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle$
- producciones no recursivas: $\langle E \rangle \rightarrow \langle T \rangle$
 - Se crea $\langle E' \rangle$
 - $\langle E \rangle \rightarrow \langle T \rangle \langle E' \rangle$
 - $\langle E' \rangle \rightarrow + \langle T \rangle \langle E' \rangle \mid \epsilon$

Para $\langle T \rangle$:

- producciones recursivas: $\langle T \rangle \rightarrow \langle T \rangle * \langle F \rangle$
- producciones no recursivas: $\langle T \rangle \rightarrow \langle F \rangle$
 - Se crea $\langle T' \rangle$
 - $\langle T \rangle \rightarrow \langle F \rangle \langle T' \rangle$
 - $\langle T' \rangle \rightarrow * \langle F \rangle \langle T' \rangle \mid \varepsilon$
 - $\langle F \rangle \rightarrow id$

Gramática resultante sin recursividad por la izquierda:

- $\langle E \rangle \rightarrow \langle T \rangle \langle E' \rangle$
- $\langle E' \rangle \rightarrow + \langle T \rangle \langle E' \rangle \mid \varepsilon$
- $\langle T \rangle \rightarrow \langle F \rangle \langle T' \rangle$
- $\langle T' \rangle \rightarrow * \langle F \rangle \langle T' \rangle \mid \varepsilon$
- $\langle F \rangle \rightarrow i$

Caso c) Factorización por la Izquierda

La factorización por la izquierda es necesaria cuando dos producciones del mismo no terminal comienzan con el mismo prefijo. Esto causa problemas en los analizadores predictivos porque no saben cuál producción elegir.

Ejemplo de gramática que requiere factorización por la izquierda:

$$\langle A \rangle \rightarrow a b c \mid a b d \mid a e$$

Las tres producciones comienzan con el prefijo común "a". Para factorizar, se extrae el prefijo común y se crea un nuevo no terminal.

Gramática original:

$$\langle A \rangle \rightarrow a b c \mid a b d \mid a e$$

- Paso 1: Extraer el prefijo común "a": $\langle A \rangle \rightarrow a \langle A' \rangle$

- Paso 2: Crear $\langle A' \rangle$ con los sufijos restantes: $\langle A' \rangle \rightarrow b\ c \mid b\ d \mid e$
- Paso 3: El prefijo "b" también es común en dos producciones, se factoriza nuevamente:
 $\langle A' \rangle \rightarrow b\ \langle A'' \rangle \mid e$
- Paso 4: Crear $\langle A'' \rangle$ con los sufijos restantes: $\langle A'' \rangle \rightarrow c \mid d$

Gramática resultante optimizada:

$$\langle A \rangle \rightarrow a\ \langle A' \rangle \mid \langle A' \rangle \rightarrow b\ \langle A'' \rangle \mid e\ \langle A'' \rangle \rightarrow c \mid d$$

Esta gramática factorizada permite a un analizador sintáctico predictivo decidir qué producción aplicar sin necesidad de retroceder.

2.4 De la Expresión al Autómata (Caso Práctico: Ajedrez)

2.4.2 Definición de un subconjunto simplificado del PGN

El lenguaje PGN (Portable Game Notation) completo incluye una gran variedad de símbolos para representar enroques (O-O), capturas (x), jaques (+), jaque mate (#) y promociones (=Q).

Para que el diseño del autómata sea abordable en este nivel, definiremos un subconjunto simplificado que solo aceptará movimientos básicos de piezas a una casilla destino, sin capturas ni modificadores adicionales.

Reglas gramaticales del subconjunto (Alfabeto y Estructura):

- Pieza (Opcional): Representada por una letra mayúscula en inglés: K (Rey), Q (Reina), R (Torre), B (Alfil), N (Caballo). Si es un movimiento de peón, se omite esta letra.
- Columna (Obligatoria): Representada por una letra minúscula de la a a la h.
- Fila (Obligatoria): Representada por un número del 1 al 8.

Ejemplos de cadenas aceptadas: Nf3, e4, Bc4, Kg1.

Ejemplos de cadenas rechazadas: Nxf3 (contiene x de captura), e4+ (contiene + de jaque), Z99 (caracteres fuera del alfabeto).

2.4.3.- Diseño de la Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD)

Expresión Regular (Regex)

Basándonos en las reglas sintácticas definidas anteriormente, la expresión regular que denota este lenguaje simplificado es:

$$^{(K|Q|R|B|N)?[a-h][1-8]}$$$

Explicación de la notación:

$^$ y $$$: Metacaracteres que indican el inicio y fin estricto de la cadena.

- $(K|Q|R|B|N)?$: Grupo de alternancia que acepta una de las letras de las piezas. El modificador: hace que este grupo sea opcional (permitiendo movimientos de peón).
- $[a-h]$: Clase de caracteres que acepta exactamente una letra minúscula en ese rango.
- $[1-8]$: Clase de caracteres que acepta exactamente un dígito en ese rango.

Autómata Finito Determinístico (AFD)

El AFD equivalente a la Expresión Regular requiere 4 estados para reconocer el lenguaje. (Nota: Puedes dibujar los círculos en tu informe basándote en esto).

- q_0 (Estado Inicial): Máquina en reposo esperando el primer carácter.
- q_1 (Pieza leída): Se alcanza si el primer símbolo ingresado es una letra mayúscula de pieza.
- q_2 (Columna leída): Se alcanza al leer la letra de la columna. Puede llegarse directo desde q_0 (si es un movimiento de peón) o desde q_1 .
- q_3 (Estado Final): Se alcanza exclusivamente al leer el número de la fila. Si la cadena termina al llegar a este estado, la palabra es aceptada.

Transiciones:

- $q_0 \xrightarrow{[K, Q, R, B, N]} q_1$
- $q_0 \xrightarrow{[a, b, c, d, e, f, g, h]} q_2$
- $q_1 \xrightarrow{[a, b, c, d, e, f, g, h]} q_2$
- $q_2 \xrightarrow{[1, 2, 3, 4, 5, 6, 7, 8]} q_3$

2.4.1.- Planteamiento y Ejecución del Autómata

Para plantear y ejecutar este autómata de forma tangible, se presenta un pequeño script computacional (en lenguaje Python) que simula el motor del autómata utilizando la Expresión Regular diseñada.

Código de ejecución (Simulador del AFD):

```
import re

def validar_pgn_basico(movimiento):
    patron_afd = r'^(K|Q|R|B|N)?[a-h][1-8]$$'
    if re.match(patron_afd, movimiento):
        return f"[ACEPTADO] La cadena '{movimiento}' es un movimiento válido."
    else:
        return f"[RECHAZADO] La cadena '{movimiento}' tiene errores de sintaxis."

movimientos = ["Nf3", "e4", "Bc4", "Kg1", "Nxf3", "e4+", "Z99", "ReyA1"]

for mov in movimientos:
    print(validar_pgn_basico(mov))
```

Salida de la ejecución:

```
[ACEPTADO] La cadena 'Nf3' es un movimiento válido.
[ACEPTADO] La cadena 'e4' es un movimiento válido.
[ACEPTADO] La cadena 'Bc4' es un movimiento válido.
[ACEPTADO] La cadena 'Kg1' es un movimiento válido.
[RECHAZADO] La cadena 'Nxf3' tiene errores de sintaxis.
[RECHAZADO] La cadena 'e4+' tiene errores de sintaxis.
[RECHAZADO] La cadena 'Z99' tiene errores de sintaxis.
[RECHAZADO] La cadena 'ReyA1' tiene errores de sintaxis.

for mov in movimientos:
    print(validar_pgn_basico(mov))
```

Conclusión

Se ha recorrido el camino que va desde los símbolos más elementales hasta la construcción de dispositivos formales capaces de generar y reconocer lenguajes. Se partió de la noción de alfabeto y palabra, se comprendió la gramática como un mecanismo generador y se exploró la Jerarquía de Chomsky como un mapa que ordena la complejidad computacional. Cada concepto, por más abstracto que pareciera, encontró su reflejo en un caso práctico: desde las bases del ADN hasta la notación del ajedrez, pasando por la corrección de errores estructurales en las gramáticas.

Se modeló un lenguaje de dibujo sobre un alfabeto genético, demostrando que incluso con un conjunto reducido de símbolos es posible construir figuras complejas mediante derivaciones sucesivas. Se enfrentaron y corrigieron patologías gramaticales como la ambigüedad, la recursividad por la izquierda y la falta de factorización, problemas reales que amenazan la estabilidad de cualquier compilador si no se atienden a tiempo. Y se diseñó un autómata finito para reconocer movimientos básicos de ajedrez, conectando la teoría de los lenguajes regulares con una aplicación concreta y cotidiana.

Cada uno de estos ejercicios no fue un fin en sí mismo, sino una pieza de un rompecabezas más grande: la comprensión de que todo lenguaje formal, desde el más sencillo hasta el más poderoso, puede ser descrito, generado y procesado con las herramientas adecuadas. La teoría de lenguajes formales no es un conjunto de reglas abstractas e inconexas, sino el esqueleto lógico que sostiene gran parte de la ingeniería de software moderna: los analizadores léxicos, los parsers, los validadores de formatos y hasta los motores de búsqueda de patrones.

Finalmente, queda demostrado que dominar estos fundamentos no es un lujo académico ni un mero requisito de una materia, sino una necesidad práctica para quien aspire a construir herramientas que interpreten, transformen o den sentido al lenguaje en el que las máquinas y los humanos se comunican. Sin esta base, cualquier intento de diseñar un compilador, un intérprete o un procesador de lenguaje estará condenado a la fragilidad y al error. Con ella, en cambio, se adquiere la capacidad no solo de usar lenguajes, sino de crearlos.

Referencias

- Colaboradores de Wikipedia. (2024). Jerarquía de Chomsky. Wikipedia, la enciclopedia libre. https://es.wikipedia.org/wiki/Jerarqu%C3%ADa_de_Chomsky
- Colaboradores de Wikipedia. (2024). Gramática libre de contexto. Wikipedia, la enciclopedia libre. https://es.wikipedia.org/wiki/Gram%C3%A1tica_libre_de_contexto
- StudySmarter. (2023). Jerarquía de Chomsky: Teoría de la Computación. Plataforma de aprendizaje StudySmarter. <https://www.studysmarter.es/resumenes/informatica/teoria-de-la-computacion/jerarquia-de-chomsky/>
- Colaboradores de Wikipedia. (2024). Compilador. Wikipedia, la enciclopedia libre. <https://es.wikipedia.org/wiki/Compilador>
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compiladores: principios, técnicas y herramientas. Pearson Educación. <https://books.google.co.ve/books?id=O98zQwAACAAJ>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación. Pearson Educación. <https://books.google.co.ve/books?id=b56QOAAACAAJ>
- Kelley, D. (1995). Teoría de Autómatas y Lenguajes Formales. Prentice Hall. <https://books.google.co.ve/books?id=oD1WPgAACAAJ>